# PRODUCT REQUIREMENT DOCUMENT (PRD)

## UMHackathon 2026

umhackathon@um.edu.my

---

© UMHackathon 2026

Email: umhackathon@um.edu.my

**Table of Contents**


1. Project Overview 3
2. Background & Business Objective 3
3. Product Purpose 5
4. System Functionalities 6
5. User Stories & Use Cases 11
6. Features Included (Scope Definition) 13
7. Features Not Included (Scope Control) 14
8. Assumptions & Constraints 15
9. Risks & Questions Throughout Development 17


*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*

**Project Name:** ChainLogic AI

**Version:** 1.0 (Hackathon Prototype)

## 1. Project Overview

- **Problem Statement:** Automotive SMEs in Malaysia and globally operate supply chains that are vulnerable to sudden, high-stakes disruptions — factory fires, customs holds, and foundry delays — all arriving as unstructured supplier emails. Without an intelligent decision layer, supply chain managers rely on intuition or slow manual analysis to respond, often incurring catastrophic daily downtime penalties (e.g., $45,000/day per delayed ADAS line) or triggering panic-driven over-ordering that amplifies the Bullwhip Effect across the entire supply network.
- **Target Domain:** AI for Economic Empowerment & Decision Intelligence — specifically, supply chain decision-support for automotive manufacturing SMEs.
- **Proposed Solution Summary:** ChainLogic AI is a closed-loop agentic decision intelligence system. It ingests unstructured supplier disruption emails, autonomously extracts the affected component SKU using Z.AI's GLM, queries a live ERP database for real-time inventory and penalty data, and generates mathematically justified trade-off options with dollar-level financial impact calculations. The optimal action is then executed directly back into the master ERP database, closing the loop without human re-entry. If the Z.AI GLM component is removed, the system is incapable of interpreting unstructured supplier emails or generating any of the financial trade-off options — reducing it to a static database viewer.

## 2. Background & Business Objective

© UMHackathon 2026
Email: umhackathon@um.edu.my

- **Background of the Problem:** Automotive SMEs manage highly interdependent supply chains where a single delayed component can halt an entire production line. The existing response workflow is fundamentally broken: a supplier sends an unstructured email, a supply chain manager reads it, manually checks inventory in a spreadsheet, calls three suppliers for quotes, and eventually makes a gut-feel decision hours or days later. During this window, downtime penalties compound at rates of $12,500 to $85,000 per day per project.

  The problem is structurally twofold:

  1. Data Fragmentation: Critical information lives in two incompatible forms — unstructured (supplier emails, force majeure notices) and structured (ERP inventory rows, penalty contracts). No single tool bridges them.

  2. Analytical Capability Gap: SMEs lack the data science resources of Tier-1 OEMs. Calculating and comparing three competing financial scenarios in real time requires a level of quantitative reasoning that human analysts struggle to produce under pressure.

- **Importance of Solving This Issue:**
  - Direct Economic Impact: A 24-hour improvement in disruption response time on a $45,000/day penalty contract saves $45,000 per incident. Across a portfolio of production jobs, this compounds annually into six-figure savings for a single SME.
  - Eliminating the Bullwhip Effect: Panic-driven over-ordering in response to a disruption is statistically shown to create cascading inventory imbalances across the supply tier. A recommendation engine that quantifies "do not over-order" as a strategy with a specific dollar value prevents this.
  - Democratizing Decision Intelligence: Tier-1 automotive firms have internal quant teams and bespoke ERP analytics. ChainLogic AI packages an equivalent capability for SMEs who cannot afford it.

© UMHackathon 2026

- **Strategic Fit / Impact:** ChainLogic AI directly aligns with Malaysia's National Industry 4.0 Policy and the UMHackathon 2026 domain of AI for Economic Empowerment. It demonstrates how Z.AI's GLM — a model with strong multilingual and domain-agnostic reasoning capability — can serve as a universal decision layer on top of any structured data source, applicable not just to automotive but to any asset-intensive SME sector.

## 3. Product Purpose

**3.1. Main Goal of the System:** To function as an always-on, AI-powered Supply Chain Decision Engine: a system that transforms raw, unstructured disruption signals into structured, financially quantified action recommendations and executes the chosen strategy autonomously within the ERP database — reducing decision latency from hours to under 30 seconds.

**3.2. Intended Users (Target Audience):**

| Role | Pain Point Addressed |
|---|---|
| Supply Chain Manager (Primary) | Overwhelmed by fragmented disruption data, no time for quantitative analysis under pressure |
| Operations Director | Needs visibility into financial exposure and strategic optionality without waiting for analyst reports |
| Procurement Officer | Must identify alternative sourcing options quickly without tribal knowledge dependency |
| SME Founder / CEO | Requires a single pane of glass to understand risk exposure and approve autonomous ERP updates |
| Production Planner | Needs to understand the impact of a missing component on the production schedule in real time |

© UMHackathon 2026

## 4. System Functionalities

**4.1. Description:** ChainLogic AI operates as a three-stage agentic pipeline. In Stage 1 (Ingest), the system receives an unstructured supplier email from the corporate inbox UI. In Stage 2 (Reason), Z.AI's GLM performs two sequential reasoning acts: first, entity extraction to identify the affected SKU, and second, financial trade-off generation against the structured ERP context. In Stage 3 (Execute), the user approves and the system autonomously writes the new operational state directly to the SQLite ERP master database, closing the decision loop.

**4.2. Key Functionalities:**

- **Agentic SKU Extraction:** The Z.AI GLM reads a free-form supplier email and returns only the part number (e.g., `AE-V8-SENS`) with no manual field-mapping required. A regex fallback provides resilience if the model is rate-limited.
- **Live ERP Fusion:** Real-time inventory stock levels, unit costs, required quantities, and daily downtime penalty rates are fetched from the SQLite database and injected into the Z.AI GLM prompt as structured context, grounding the model in factual data and eliminating hallucinated numbers.
- **Financial Trade-Off Analysis:** Z.AI's GLM generates 2–3 named options (e.g., Air Freight via Secondary Supplier, Ground Freight, Schedule Reschedule) with precise net financial impact figures, a computation formula, and a math breakdown string — all rendered as interactive cards in the UI.
- **GLM Recommendation Engine:** Beyond listing options, the GLM selects a primary recommendation with an explainability statement that cites specific dollar amounts, mentions the Bullwhip Effect risk where applicable, and justifies the strategic choice.
- **Closed-Loop ERP Execution:** On user approval, the `/api/execute-decision` endpoint updates the `production_jobs` table (tagging the action taken and zeroing the downtime penalty) and adjusts `inventory_parts` unit cost if a financial impact

© UMHackathon 2026

was incurred — providing a permanent, auditable ERP record of the AI's decision.

- **Safe State Detection**: If current inventory already meets or exceeds the required quantity, or the penalty is zero, the system short-circuits the AI call and returns a `SAFE` state instantly, avoiding unnecessary token consumption.

- **Corporate Inbox Interface:** A Gmail-styled email inbox surfaces realistic supplier disruption scenarios (customs holds, factory fires, foundry delays) and routes selected emails directly into the analysis pipeline with a single click.
- **Data Pipeline Transparency Viewer:** An expandable code-style viewer shows the exact SQL query result — current stock, unit cost, daily penalty rate — that was fused with the email text before being sent to Z.AI GLM, providing full auditability of the AI's data context.

### 4.3. AI Model & Prompt Design

#### 4.3.1. Model Selection:

Model Used: Z.AI GLM (`ilmu-glm-5.1`) via `https://api.ilmu.ai/v1/chat/completions`

Justification: Z.AI's GLM was selected as the core reasoning engine for the following capability-specific reasons:

| Capability Required | Why Z.AI GLM |
|---|---|
| Unstructured text interpretation | Extracts structured entities (SKUs) from free-form supplier emails without fine-tuning |
| Context-aware reasoning | Accepts a system prompt defining a domain expert persona and adheres to it consistently |

| | |
|---|---|
| Strict JSON schema output | Reliably returns schema-conformant JSON when instructed, enabling direct API-to-UI data flow |
| Financial quantitative reasoning | Performs multi-variable financial calculations (penalty × days, sourcing premium, expedite fees) within the reasoning context |
| OpenAI-compatible API | Enables integration via standard `Authorization: Bearer` headers and the `/chat/completions` endpoint using Python's `urllib`, requiring no additional SDKs |

Critically, Z.AI GLM enables the dual-agent architecture (a separate extractor call and a separate analyser call) within a single model, reducing system complexity while maintaining separation of concerns.

#### 4.3.2. **Prompting Strategy**

Multi-step agentic prompting (two-call pipeline), with zero-shot prompting for the extraction step and structured system-prompt prompting for the analysis step.

Step 1, SKU Extraction (Zero-shot):

"Extract the specific part number/SKU from this text. Output ONLY the SKU string, no other words."

Zero-shot was chosen because SKU extraction is a deterministic, single-entity extraction task. The model does not need examples — the instruction is unambiguous and the output space is constrained. Adding few-shot examples would increase token cost per call without improving accuracy for this task.

© UMHackathon 2026

Step 2, Crisis Analysis (Structured System Prompt):

A full CHAINLOGIC_SYSTEM_PROMPT is injected as the system role, establishing the model's identity ("ChainLogic AI, an expert Supply Chain Decision Engine"), defining the exact JSON output schema, and providing the critical instruction to return raw JSON only. The user prompt then fuses the unstructured email with the structured ERP data in a single message.

This hybrid approach was chosen because the analysis task is complex, multi-output, and must conform to a specific schema. The system prompt acts as a persistent reasoning frame that survives across the length of the user turn, ensuring the model always remains in its "supply chain analyst" role.

#### 4.3.3. **Context & Input Handling**

The system accepts two input types:

| **Input Type** | **Source** | **Size Management** |
|---|---|---|
| Unstructured email text | User-pasted or inbox-selected | No hard truncation. Emails are typically <= 1,000 tokens. If an unusually long document is submitted, the GLM's native context window handles it. |
| Structured ERP context | SQLite query result (formatted string) | Fixed and minimal: SKU \| Stock: {N} \| Penalty: ${N}/day - typically <= 20 tokens. This is intentional to keep the ERP context compact and prevent the structured data from overwhelming the unstructured signal. |

**Combined prompt size** for a typical session is estimated at ~1,200-1,800 tokens input + ~600 tokens output = ~2,000 tokens per analysis call. The SKU extraction call adds ~100 tokens.

© UMHackathon 2026

If an email contains no identifiable SKU pattern (verified by both the GLM and a regex fallback), the system returns a graceful error: "Z.AI could not identify a valid component SKU in the provided text." — no API call is wasted on unparseable input.

4.3.4. **Fallback & Failure Behavior:**

The system implements a three-layer resilience architecture:

| **Layer** | **Trigger** | **Response** |
|---|---|---|
| **Layer 1 — Regex Fallback** | Z.AI SKU extraction call fails (HTTP error, timeout, rate limit) | Python re.search(r'[A-Z0-9]{2,}-[A-Z0-9]{2,}-[A-Z0-9]{2,}', ...) extracts the SKU from the raw email text. Logs: "Extraction AI failed, falling back to Regex". |
| **Layer 2 —Hardcoded Mock Response** | Z.AI analysis call (analyze-crisis) returns an HTTP error | A pre-written, structurally valid JSON response with two plausible trade-off options (Air Freight and Schedule Reschedule) is returned. The live ERP data is still injected into this mock, so inventory figures remain accurate even in the fallback state. |
| **Layer 3 —Structured Error Response** | SKU not found in ERP database after successful extraction | Returns {"error": "SKU '{target_sku}' extracted from text, but it does not exist in the Enterprise Database master records."} — rendered as a clear user-facing message. |

There is no human escalation path in the current prototype scope. Hallucinated or off-schema JSON responses from the GLM are caught by a json.loads() try/except block, which triggers Layer 2.

© UMHackathon 2026

## 5. User Stories & Use Cases

- **User Stories:**

| # | Role | Story | Acceptance Criteria |
|---|---|---|---|
| US-01 | Supply Chain Manager | "As a supply chain manager, I want to forward a supplier disruption email into ChainLogic AI so that I receive a structured, dollar-quantified response in under 60 seconds without building a spreadsheet." | System returns a structured JSON response with >= 2 trade-off options and a primary recommendation within 30 seconds. |
| US-02 | Supply Chain Manager | "As a supply chain manager, I want the AI to automatically identify which SKU is at risk from the email so I don't have to manually look it up." | System correctly identifies the SKU from the email body with no user input beyond selecting the email. |
| US-03 | Operations Director | "As an operations director, I want to see the exact database rows the AI used in its analysis so I can trust the financial figures it produces." | The Data Pipeline Viewer panel shows the exact inventory values (stock, cost, penalty) fetched from SQLite. |
| US-04 | Procurement Officer | "As a procurement officer, I want to see multiple response options with their financial trade-offs so I can compare them before committing." | UI displays >= 2 trade-off cards with net_financial_impact, formula, and math breakdown per option. |
| US-05 | SME Founder | "As a founder, I want to execute the AI's recommended decision with one click and have it immediately reflected in the ERP database." | Clicking "Execute Decision" updates production_jobs.daily_downtime_penalty to 0 and tags the project |

© UMHackathon 2026
Email: umhackathon@um.edu.my

| # | Role | Story | Acceptance Criteria |
|---|---|---|---|
| | | | name with the chosen action in SQLite. |
| US-06 | Production Planner | "As a production planner, I want the system to tell me immediately if stock levels are already safe, so I'm not alarmed unnecessarily." | If current_stock >= required_quantity, system returns status SAFE without making a Z.AI GLM call. |

- **Use Cases (Main Interactions):**

**UC-01: Disruption Email → Financial Decision (Primary Flow)**

1. User opens the Corporate Inbox and selects an unread disruption email.
2. User clicks "Analyze with ChainLogic AI".
3. System calls Z.AI GLM (Step 1): extracts the affected SKU from the email body.
4. System queries SQLite: fetches live inventory, penalty, and required quantity for that SKU.
5. Safe State Check: if stock is sufficient, return `SAFE` status (go to UC-03). Otherwise, continue.
6. System calls Z.AI GLM (Step 2): sends the email text + structured ERP context; receives a JSON response with trade-off options and a primary recommendation.
7. Dashboard renders the Crisis Banner, Data Pipeline Viewer, Trade-Off Cards, and GLM Recommendation.
8. User reviews the recommended option, clicks "Execute Decision".
9. Backend updates SQLite ERP tables. UI confirms: "✓ ERP Updated".

**UC-02: Re-Run Analysis Flow**

1. User is on the Dashboard view and edits the email text in the textarea.

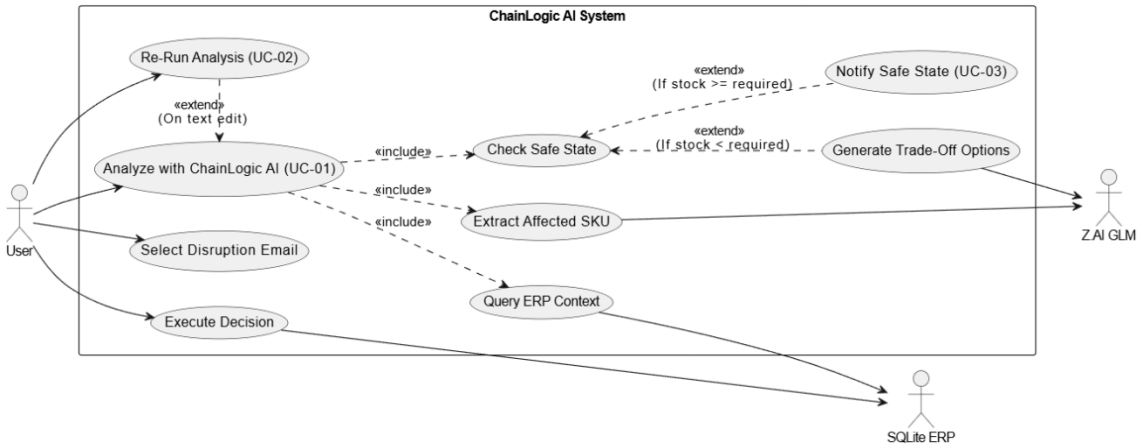
© UMHackathon 2026

2. User clicks "Re-Run Analysis".
3. System repeats UC-01 from Step 3 with the modified text.
4. New results replace the previous output on the same screen.

**UC-03: Safe State Notification**

1. UC-01 Step 5 determines `current_stock ≥ required_quantity`.
2. System immediately returns a `SAFE` status response (no GLM analysis call).
3. Dashboard renders a green Safe State banner: `"Stock levels are sufficient (N/M) — no action required."`

**Use Case Diagram**




## 6. Features Included (Scope Definition)

- Corporate Inbox Interface: A Gmail-styled inbox listing 5 realistic disruption email scenarios (customs holds, factory fires, foundry delays, shipment confirmations) with unread/read indicators and email detail view.
- Agentic Two-Step Z.AI GLM Pipeline: A dedicated SKU extraction call followed by a financial analysis call, each with distinct prompts and output validation.
- Live SQLite ERP Database: A normalised, auto-seeded relational database (suppliers, categories, inventory_parts, production_jobs, job_requirements) that is read at analysis time and written to at execution time.

© UMHackathon 2026
Email: umhackathon@um.edu.my

- Financial Trade-Off Cards: Interactive UI cards displaying option ID, action description, net financial impact (with positive/negative styling), Z.AI GLM reasoning, and a collapsible computation breakdown (formula + math).
- Closed-Loop ERP Execution: The ability to autonomously update production_jobs (penalty zeroed, action tagged) and inventory_parts (unit cost adjusted) via the /api/execute-decision endpoint.
- Data Pipeline Transparency Viewer: An expandable, code-styled panel showing the exact SQL result injected into the GLM prompt.
- Safe State Short-Circuit: Automated detection that skips the GLM call when inventory levels are sufficient.
- Three-Layer Fallback Architecture: Regex SKU extraction backup, hardcoded mock response, and structured error messages.
- RESTful FastAPI Backend: Two primary endpoints (/api/analyze-crisis and /api/execute-decision) with CORS enabled for the Next.js frontend.

**7. Features Not Included (Scope Control)**

- **Real Supplier Email Integration:** There is no live email protocol (IMAP/SMTP) connection. The Corporate Inbox uses mock email data defined in InboxView.tsx. Real email parsing is out of scope.
- **Multi-SKU Disruption Handling:** The current pipeline processes one SKU per email. Emails referencing multiple affected components in a single disruption are not supported; the system extracts the first identifiable SKU.

© UMHackathon 2026

- **Historical Analytics & Trend Dashboards:** There is no time-series analysis, disruption history log, or predictive forecasting module. Analysis is purely reactive to the current trigger.
- **User Authentication & Multi-User Support:** There is no login system, role-based access control, or session management. The application is single-user and open for hackathon demonstration purposes.
- **Production Deployment & Scalability:** The backend runs on a local Uvicorn development server. Load balancing, containerisation (Docker/Kubernetes), and cloud deployment are out of scope.
- **Alternative LLM Configuration UI:** The AI provider, model name, and API key are environment-variable-only. There is no settings panel for non-technical users to switch models.
- **Automated Supplier Outreach:** The system recommends contacting a secondary supplier but does not generate or send any communications on the user's behalf.
- **Real ERP System Integration:** The SQLite database is a normalised mock. Integration with actual ERP systems (SAP, Oracle, Odoo) is not included.

## 8. Assumptions & Constraints

- **LLM Cost Constraint:**

  **Estimated token cost per average user session:** - SKU Extraction call: ~150 tokens input + 10 tokens output = 160 tokens

  - Crisis Analysis call: ~1,600 tokens input + 600 tokens output = 2,200 tokens - **Total per session: ~2,360 tokens**

  At Z.AI's GLM pricing, this is estimated at < $0.01 USD per session, making the system economically viable at scale.

  **Key cost-control design decision:** The Safe State short-circuit (if current_stock ≥ required_quantity) skips the second (more expensive) GLM call entirely. For a supply chain where most components are not in crisis at any given time, this could eliminate 70â€“80% of analysis-stage API calls. Additionally, the ERP context string injected into

the prompt is deliberately minimised (a single formatted line, â‰¤25 tokens) rather than dumping entire table dumps, keeping input tokens controlled.

- **Technical Constraints**
  - The system requires Python 3.9+ and Node.js 18+ to run locally. No containerised deployment is provided.
  - The SQLite database is file-based (chainlogic_erp.db). Concurrent write operations from multiple simultaneous users could cause lock contention.
  - API calls to Z.AI GLM are synchronous, blocking the FastAPI response until the model returns. For long model responses, the frontend loading state may persist for 10â€“30 seconds.
  - The frontend calls http://localhost:8000 directly (hardcoded). Changing the backend host requires a code change.

- **Performance Constraints**
  - Z.AI GLM response time is not guaranteed. Network latency and model load can cause response times to vary from 5 to 30 seconds per call.
  - The dual-call agentic pipeline (extraction + analysis) adds latency compared to a single-call approach. This trade-off was accepted to maintain clean separation of concerns and enable the regex fallback layer.
  - The hardcoded mock fallback response is returned synchronously with no delay when the API fails, ensuring the UI is never left in a permanent loading state.

- **User Input**
  - The Z.AI GLM cannot generate meaningful trade-off options without both an unstructured trigger text AND a valid, database-matched SKU. Either component alone is insufficient.

© UMHackathon 2026

- The system does not operate autonomously. All ERP execution steps require explicit user approval via the "Execute Decision" button. The AI recommends; the human decides.

- The system is a prototype. ERP updates are not transactional (no rollback mechanism) and should not be connected to a production database without additional safeguards.

## 9. Risks & Questions Throughout Development

**Risk 1: SKU Extraction Accuracy**

**Risk:** Z.AI GLM may occasionally fail to extract a SKU from an email that contains it (false negative) or extract the wrong alphanumeric string (false positive), especially for non-standard part number formats.

**Mitigation:** A secondary regex pattern [A-Z0-9]{2,}-[A-Z0-9]{2,}-[A-Z0-9]{2,} serves as the fallback. Future versions should include a validation step: after GLM extraction, the SKU is immediately checked against the database, and if not found, the regex fallback is tried before returning an error.

**Risk 2: GLM JSON Schema Hallucination**

**Risk:** Z.AI GLM may return structurally invalid JSON, JSON with missing required keys, or markdown-wrapped JSON (e.g., wrapped in ```json ``` blocks) that breaks the json.loads() parser.

**Mitigation:** The backend implements a JSON sanitisation step that strips markdown code fences before parsing. A try/except block catches all json.loads errors and triggers the hardcoded mock fallback. A stricter future mitigation would be to use structured output / function calling modes if the Z.AI API supports them.

**Risk 3: Financial Figure Accuracy**

**Risk:** The dollar amounts in the trade-off options are generated by the GLM from the ERP context prompt. While the ERP data (penalty rate, stock level) is factual, the GLM's *computation* of multi-step financial scenarios could contain arithmetic errors.

**Mitigation:** The baseline_impact field in the response is always overwritten by a Python-computed string using actual database values, ensuring the headline figures are accurate. The computation_breakdown.math field (a GLM-generated string) is presented as "GLM Breakdown" in the UI, signalling to users that it is model-generated reasoning, not a certified audit figure.

**Risk 4: Bullwhip Effect in Recommendations**

**Risk:** The GLM may recommend over-ordering (e.g., Option B: "Order 200% of required quantity from secondary supplier") without flagging the downstream inventory imbalance risk this creates.

**Mitigation:** The system prompt explicitly instructs the model: *"calculate financial trade-offs and recommend actions to prevent production downtime and mitigate the 'Bullwhip Effect' (over-ordering out of panic)."* The explainability field requirement asks the model to mention Bullwhip Effect avoidance where applicable, embedding this risk awareness into every recommendation cycle.

**Risk 5: Data Staleness**

**Risk:** The ERP database is seeded with static mock data. After an "Execute Decision" action zeroes a penalty, re-running the analysis on the same email will return a SAFE state, which is correct behaviour — but may confuse demo evaluators who expect a crisis state on replay.

**Mitigation:** Document this as intended closed-loop behaviour. The demo should be scripted to show the analysis *before* execution, then demonstrate the ERP update, and finally show the SAFE state on re-analysis as a proof of system correctness.

**Open Question: Multi-Language Support**

© UMHackathon 2026

**Question:** Supplier emails in real-world Malaysian automotive supply chains may arrive in Bahasa Malaysia or Mandarin Chinese. Does Z.AI GLM's multilingual capability extend to accurate SKU extraction from non-English email text?

**Current Status:** Not tested. This is a risk for any real-world deployment targeting Malaysian SMEs and warrants validation in a follow-up testing cycle.

*© UMHackathon 2026*

*Email: umhackathon@um.edu.my*